Лабораторная работа №3
Базовые растровые алгоритмы

Храмцов Родион
31 октября 2026 г.

1. Цель работы

Изучить базовые алгоритмы растровой развёртки (растеризации) отрезков и окружностей: понять, как непрерывная геометрическая фигура с целочисленными параметрами приводится к множеству пикселей дискретной сетки; реализовать приложение с графическим интерфейсом, наглядно иллюстрирующее работу алгоритмов; сравнить алгоритмы между собой по точности, составу вычислений и времени работы. Обязательно реализованы четыре алгоритма:

- пошаговый алгоритм (по уравнению прямой $y = kx + b$);
- алгоритм цифрового дифференциального анализатора (ЦДА);
- алгоритм Брезенхема для отрезка;
- алгоритм Брезенхема для окружности.

Дополнительно (задания на повышенную оценку) реализованы алгоритм Кастл–Питвея для отрезка и сглаживание отрезков по Ву.

Для каждого алгоритма в работе требуется: (1) строго обосновать математически, почему он выбирает именно эти пиксели; (2) показать сопроводительные вычисления на конкретном примере; (3) количественно оценить точность и согласованность с остальными алгоритмами; (4) измерить время работы своей реализации; (5) сформулировать особенности, достоинства, недостатки и границы применимости. Интерфейс должен выводить систему координат, оси, линии сетки и подписи, позволять менять масштаб и объяснять, как целочисленные координаты привязаны к дискретной сетке.

Критерии корректности, по которым проверялись алгоритмы: (а) на отрезок с длиной $L = \max(|\Delta x|, |\Delta y|)$ приходится ровно $L + 1$ пиксель, первый и последний совпадают с концами, соседние пиксели соприкасаются (8-связность); (б) отклонение каждого пикселя от идеальной линии вдоль второстепенной оси не превышает $1/2$; (в) для окружности отклонение $\left|\sqrt{x^2 + y^2} - R\right|$ не превышает $1/2$, фигура симметрична относительно осей и диагоналей.

2. Математика и алгоритмы

2.1. Дискретная сетка и постановка задачи

Растр — прямоугольная сетка клеток. Пиксель с целыми координатами (x, y) — это клетка, *центр* которой лежит в точке (x, y) плоскости; границы клеток проходят по полуцелым значениям $x \pm \frac{1}{2}$, $y \pm \frac{1}{2}$. Такая привязка симметрична (все четыре стороны клетки равноправны) и позволяет считать расстояние от идеальной фигуры до пикселя как расстояние до его центра.

Для отрезка с целыми концами (x_1, y_1), (x_2, y_2) обозначим $\Delta x = x_2 - x_1$, $\Delta y = y_2 - y_1$, $L = \max(|\Delta x|, |\Delta y|)$. Ось, вдоль которой $|\Delta|$ больше, называется *главной*: по ней алгоритм делает ровно один шаг на пиксель; вторая ось — *второстепенная*. Требуется выбрать по одному пикселю на каждое значение главной координаты, причём ближайшему к идеальной линии по второстепенной оси. Здесь и далее подробно разбирается первый октант $0 \le \Delta y \le \Delta x$; остальные получаются заменой знаков шагов $s_x = \operatorname{sgn} \Delta x$, $s_y = \operatorname{sgn} \Delta y$ и обменом осей при $|\Delta y| > |\Delta x|$. Округление понимается как $\lfloor v + \frac{1}{2} \rfloor$ («половина вверх»): встроенный `round` языка Python округляет $0{,}5$ до чётного и для растеризации не подходит.

2.2. Пошаговый алгоритм

Прямая через концы отрезка задаётся уравнением

$$y = kx + b, \qquad k = \frac{\Delta y}{\Delta x}, \quad b = y_1 - kx_1. \tag{1}$$

Для каждого целого $x = x_1, \ldots, x_2$ вычисляется $y_i = \left\lfloor kx_i + b + \frac{1}{2} \right\rfloor$. Ошибка $\varepsilon_i = y_i - (kx_i + b)$ по построению лежит в $[-\frac{1}{2}, \frac{1}{2}]$ и не накапливается: каждое y считается заново по формуле. Если $|\Delta y| > |\Delta x|$, то идти по x нельзя: на каждый шаг x линия поднимается более чем на одну клетку, и в столбцах появляются разрывы; поэтому роли осей меняются: $x = k'y + b'$, $k' = \Delta x / \Delta y$. Вертикальный отрезок ($\Delta x = 0$, $k = \infty$) тоже обрабатывается этой ветвью. Стоимость пикселя: умножение, сложение и округление над вещественными числами.

2.3. Алгоритм ЦДА

Запишем отрезок параметрически: $x(t) = x_1 + t\,\Delta x/L$, $y(t) = y_1 + t\,\Delta y/L$, $t \in [0, L]$. Параметр выбран так, что по главной оси координата меняется ровно на единицу за единицу t. Цифровой дифференциальный анализатор интегрирует уравнения $\dot{x} = \Delta x/L$, $\dot{y} = \Delta y/L$ методом Эйлера, который для постоянной правой части точен:

$$x_{i+1} = x_i + \frac{\Delta x}{L}, \qquad y_{i+1} = y_i + \frac{\Delta y}{L}, \qquad \text{пиксель: } \left(\left\lfloor x_i + \tfrac{1}{2} \right\rfloor, \left\lfloor y_i + \tfrac{1}{2} \right\rfloor\right). \tag{2}$$

Умножения в цикле нет, одна и та же схема работает при любом наклоне. Единственный источник погрешности — конечная разрядность: приращение $\Delta y/L$ представляется с относительной ошибкой порядка 2^{-p} (p — число бит мантиссы), и эта ошибка складывается i раз, поэтому систематическая составляющая ошибки y_i растёт линейно с номером шага; кроме того, в каждом сложении теряются младшие разряды. Численный эксперимент в разд. 5.1.4 показывает, что при одинарной точности (24 бита) отрезок длиной 10^5 пикселей искажается на десятки пикселей.

2.4. Алгоритм Брезенхема для отрезка

Идея — получить решение «выше или ниже середины между двумя кандидатами» без вещественных чисел. Пусть выбран пиксель (x_i, y_i). В следующем столбце $x_i + 1$ идеальная линия имеет ординату $y^* = y_1 + \frac{\Delta y}{\Delta x}(x_i + 1 - x_1)$, а кандидатами являются $(x_i + 1, y_i)$ и $(x_i + 1, y_i + 1)$. Расстояния от y^* до них: $d_{\text{н}} = y^* - y_i$ и $d_{\text{в}} = y_i + 1 - y^*$. Выбираем верхний, если $d_{\text{н}} \geq d_{\text{в}}$. Умножением на $\Delta x > 0$ (знак сохраняется) убираем дробь и вводим *параметр решения*

$$e_i = \Delta x\,(d_{\text{н}} - d_{\text{в}}) = 2\Delta y\,(x_i + 1 - x_1) - 2\Delta x\,(y_i - y_1) - \Delta x. \tag{3}$$

Правило: $y_{i+1} = y_i + 1$ при $e_i \geq 0$, иначе $y_{i+1} = y_i$. Начальное значение при $x_0 = x_1$, $y_0 = y_1$:

$$e_0 = 2\Delta y - \Delta x. \tag{4}$$

Рекуррентное соотношение получается вычитанием формул (3) для соседних i: x растёт на 1, поэтому первое слагаемое прибавляет $2\Delta y$, а второе вычитает $2\Delta x$ при каждом шаге по y:

$$e_{i+1} = e_i + 2\Delta y - 2\Delta x\,(y_{i+1} - y_i) = \begin{cases} e_i + 2\Delta y - 2\Delta x, & e_i \geq 0, \\ e_i + 2\Delta y, & e_i < 0. \end{cases} \tag{5}$$

Итак, цикл содержит только целочисленные сложения, вычитания и сравнение со знаком (величины $2\Delta x$, $2\Delta y$ считаются один раз). Алгоритм: (1) вычислить $\Delta x, \Delta y, s_x, s_y$; если $|\Delta y| > |\Delta x|$, поменять роли осей; (2) $e := 2\Delta y - \Delta x$, $(x, y) := (x_1, y_1)$; (3) повторить $\Delta x + 1$ раз: поставить пиксель; если $e \geq 0$, то $y \mathrel{+}= s_y$, $e \mathrel{-}= 2\Delta x$; затем $x \mathrel{+}= s_x$, $e \mathrel{+}= 2\Delta y$. При $e = 0$ (идеальная линия проходит ровно по границе двух клеток) пиксель выбирается в направлении движения по второстепенной оси; это правило разрешения «ничьих» важно для сравнения с другими алгоритмами (разд. 5.1.3).

2.5. Алгоритм Брезенхема для окружности

Окружность $F(x,y) = x^2 + y^2 - R^2 = 0$ симметрична относительно осей и диагоналей, поэтому строится одна восьмая дуга — от точки $(0, R)$ до диагонали $x = y$ (1-й октант), а остальные семь получаются отражениями $(\pm x, \pm y)$, $(\pm y, \pm x)$. В 1-м октанте наклон дуги по модулю не больше единицы, поэтому x растёт на 1 при каждом шаге, а y либо остаётся, либо уменьшается на 1. Из пикселя (x_i, y_i) кандидаты — $(x_i + 1, y_i)$ и $(x_i + 1, y_i - 1)$; решение принимается по знаку F в средней точке $M = (x_i + 1,\ y_i - \tfrac{1}{2})$:

$$d_i = F\big(x_i + 1,\ y_i - \tfrac{1}{2}\big) = (x_i + 1)^2 + \big(y_i - \tfrac{1}{2}\big)^2 - R^2. \tag{6}$$

Если $d_i < 0$, середина лежит внутри окружности, т. е. дуга проходит выше M и ближе к (x_i+1, y_i); если $d_i \geq 0$ — выбирается $(x_i + 1, y_i - 1)$. Рекуррентные формулы получаются подстановкой следующей средней точки:

$$d_i < 0: \quad d_{i+1} = F\big(x_i + 2,\ y_i - \tfrac{1}{2}\big) = d_i + 2x_i + 3, \tag{7}$$

$$d_i \geq 0: \quad d_{i+1} = F\big(x_i + 2,\ y_i - \tfrac{3}{2}\big) = d_i + 2(x_i - y_i) + 5. \tag{8}$$

Начало в $(0, R)$: $d_0 = F(1, R - \tfrac{1}{2}) = \tfrac{5}{4} - R$. Все приращения целые, поэтому d_i всегда равно целому числу плюс $\tfrac{1}{4}$; для такого числа условие $d < 0$ равносильно $d - \tfrac{1}{4} < 0$. Следовательно, можно хранить $h = d - \tfrac{1}{4}$ с начальным значением $h_0 = 1 - R$ и работать только с целыми — именно эта форма использована в программе. Умножив h на 2 и прибавив 1, получаем классическую запись Брезенхема $\Delta_0 = 3 - 2R$ с приращениями $4x + 6$ и $4(x - y) + 10$; обе формы выбирают одни и те же пиксели (проверено для $R = 1, \ldots, 300$, разд. 5.2). Цикл выполняется при $x \leq y$. Пиксели на осях ($x = 0$) и диагоналях ($x = y$) при отражении получаются дважды и удаляются как дубликаты.

2.6. Алгоритм Кастл–Питвея

В алгоритме Брезенхема решение принимается в каждом пикселе, хотя шаблон шагов периодичен. Пусть $0 < b < a$ ($a = |\Delta x|$, $b = |\Delta y|$ в 1-м октанте), $g = \gcd(a, b)$, $a' = a/g$, $b' = b/g$. Путь состоит из шагов двух видов: `x` — на пиксель вправо, `d` — по диагонали (вправо и вверх); за период длиной a' встречаются $a' - b'$ шагов `x` и b' шагов `d`. Нужно разместить их *максимально равномерно*. Такая задача решается алгоритмом Евклида, применённым к строкам. Положим $m_1 = a' - b'$, $m_2 = b'$, $s_1 =$ `x`, $s_2 =$ `d`; искомая последовательность — равномерное размещение m_1 копий s_1 и m_2 копий s_2. Пока $m_1 \neq m_2$:

- если $m_1 > m_2$: каждой копии s_2 предшествует копия s_1; заменяем $s_2 := s_1 s_2$, $m_1 := m_1 - m_2$;
- если $m_1 < m_2$: каждая копия s_1 сопровождается копией s_2; заменяем $s_1 := s_1 s_2$, $m_2 := m_2 - m_1$.

Так как $\gcd(a' - b', b') = 1$, процесс заканчивается при $m_1 = m_2 = 1$, и строка $w = s_1 s_2$ длины a' — период. Это нижнее слово Кристоффеля: j-й шаг диагональный тогда и только тогда, когда $\lfloor jb'/a' \rfloor - \lfloor (j-1)b'/a' \rfloor = 1$, то есть путь идёт *под* идеальной линией (отклонение до 1 пикселя). Чтобы получить округление к ближайшему пикселю, как у Брезенхема, где $y_i = \lfloor ib'/a' + \tfrac{1}{2} \rfloor$, период нужно циклически сдвинуть. Подстановка показывает, что при сдвиге на t позиций слово даёт $\lfloor (jb' + c)/a' \rfloor$ с $c \equiv tb' \pmod{a'}$; нужное $c = \lfloor a'/2 \rfloor$, откуда

$$t \equiv \left\lfloor \frac{a'}{2} \right\rfloor \cdot (b')^{-1} \pmod{a'}. \tag{9}$$

Итоговый путь — сдвинутое слово, повторённое g раз. В цикле рисования остаются лишь перебор символов строки и сдвиг координат, без вычисления ошибки. Метод восходит к лингвистическому описанию прямой на сетке (Бронс, 1974) и применению алгоритма Евклида (Кастл, Питвей, 1985).

2.7. Сглаживание по Ву

Идеальная линия имеет ширину в один пиксель и в каждом столбце накрывает части двух соседних клеток. Вместо выбора одной клетки закрашиваются обе, причём яркость пропорциональна близости центра клетки к линии. Пусть для столбца x ордината идеальной линии $y^* = kx + b$, $y_{\text{int}} = \lfloor y^* \rfloor$, $f = y^* - y_{\text{int}}$. Пиксели (x, y_{int}) и $(x, y_{\text{int}} + 1)$ получают яркости

$$I_{\text{н}} = 1 - f, \qquad I_{\text{в}} = f, \qquad I_{\text{н}} + I_{\text{в}} = 1. \tag{10}$$

Сумма яркостей в каждом столбце равна единице, поэтому линия имеет одинаковую яркость на всём протяжении. Кроме того, яркостный центр столбца точно совпадает с идеальной линией: $I_{\text{н}}\, y_{\text{int}} + I_{\text{в}}(y_{\text{int}} + 1) = y_{\text{int}} + f = y^*$, то есть погрешность положения становится субпиксельной (при несглаженной линии она достигает $1/2$). Для $|k| > 1$ роли осей меняются. На экране яркость передаётся прозрачностью цвета линии над белым фоном.

2.8. Сравнение вычислительной стоимости

В табл. 1 перечислены операции, которые выполняются *на каждый пиксель* главной оси, и подготовка перед циклом.

Таблица 1: Состав вычислений алгоритмов

Алгоритм	В цикле (на пиксель)	Тип чисел	Подготовка
Пошаговый	умножение, сложение, округление	вещественные	деление (k), умножение (b)
ЦДА	2 сложения, 2 округления	вещественные	2 деления
Брезенхем (отрезок)	2–3 сложения/вычитания, сравнение	целые	$2\Delta x$, $2\Delta y$, e_0
Брезенхем (окружность)	2–3 сложения, сравнение; 8 отражений	целые	$d_0 = 1 - R$
Кастл–Питвей	выбор шага по символу строки, 1–2 инкремента	целые	НОД, $O(\log a)$ склеек строк, сдвиг
Ву	сложение, $\lfloor\cdot\rfloor$, 2 пикселя с яркостью	вещественные	деление (k)

3. Параметры окружения и используемый стек

- Язык программирования: Python 3 (тестирование на версии 3.12).
- Графический интерфейс: PyQt6 (`QWidget` с собственной отрисовкой через `QPainter`, `QTableWidget`, `QSplitter`).
- Алгоритмы растеризации написаны с использованием только стандартной библиотеки (`math`, `time.perf_counter`); встроенные функции рисования линий и окружностей для получения пикселей не используются. Средствами `QPainter` нарисованы лишь красные «идеальные» фигуры для сравнения, а сами пиксели закрашиваются как квадраты по результатам собственных алгоритмов.
- NumPy и Matplotlib использованы только для статистики и рисунков данного отчёта, приложению они не нужны.
- Установка: `pip install PyQt6`. Запуск: `python raster_lab3.py`. Сборка исполняемого файла: `pyinstaller --onefile --windowed raster_lab3.py`.

Структура программы: функции `step_by_step`, `dda`, `bresenham`, `bresenham_circle`, `castle_pitteway`, `wu` возвращают список пикселей, заголовки таблицы вычислений и пояснение; при передаче аргумента `log` они дополнительно заполняют таблицу шагов. Класс `Canvas` отвечает за сетку, оси, масштаб и закраску, `MainWindow` — за панель управления, таблицы и замеры времени.

4. Формирование тестового набора и интерфейс

4.1. Тестовый набор

Входные данные алгоритмов — не изображения, а геометрические примитивы, поэтому тестовый набор состоит из специально подобранных отрезков и окружностей (табл. 2). Каждая группа проверяет отдельную особенность.

Таблица 2: Тестовый набор примитивов

Группа	Примитивы	Для чего
Частные случаи	горизонталь, вертикаль, диагональ $45°$, вырожденный отрезок (точка)	ветвления, $L = 0$
Все октанты	16 отрезков из начала координат в точки $(\pm 8, \pm 3)$, $(\pm 3, \pm 8)$, $(\pm 8, \pm 8)$, $(\pm 8, 0)$, $(0, \pm 8)$	приведение октантов
Расчётный пример	$(-6, -3) \to (8, 5)$, $\Delta x = 14$, $\Delta y = 8$, НОД $= 2$	сопроводительные вычисления
Случай «ничьей»	$(0, 0) \to (10, -5)$, $k = -0{,}5$	разрешение $e = 0$
Случайные отрезки	3000 отрезков, концы в $[-100, 100]^2$, генератор с фиксированным зерном	статистика точности
Длинные отрезки	$(0, 0) \to (n, \lfloor 0{,}6n \rfloor)$, $n = 10; 10^2; 10^3; 10^4$	время работы
Дрейф ЦДА	$(0, 0) \to (n, \lfloor 0{,}37n \rfloor + 1)$, $n = 10^3 \dots 10^6$	накопление ошибки
Окружности	$R = 0, 1, 2, 3, 8$ (пример), $R = 1, \dots, 300$ (статистика), $R = 10 \dots 10^4$ (время)	симметрия, точность

4.2. Метрики

Так как для каждого примитива известна идеальная фигура, применялись количественные метрики:

- число пикселей, совпадение концов и 8-связность (критерий (а) из разд. 1);
- отклонение ε пикселя от идеальной линии вдоль второстепенной оси, его максимум и среднее модуля, а также максимальное перпендикулярное расстояние ρ до линии;
- доля отрезков и доля пикселей, не совпавших с результатом Брезенхема; для всех расхождений проверялось, лежит ли идеальная линия ровно на границе двух клеток (точная проверка в рациональных числах);
- симметричность: совпадает ли множество пикселей при построении от P к Q и от Q к P;
- накопление ошибки ЦДА: число пикселей, отличающихся от точного округления, при float32 и float64;
- для окружности: максимальное $\left|\sqrt{x^2 + y^2} - R\right|$, совпадение с классической формой $\Delta_0 = 3 - 2R$ и с «эталоном» (ближайший пиксель в каждом столбце 1-го октанта);

- для сглаживания: сумма яркостей в столбце и смещение яркостного центра от идеальной линии;
- время работы вызова функции растеризации без отрисовки и без ведения журнала.

4.3. Интерфейс

Окно состоит из левой панели управления, поля с сеткой и нижних вкладок. На панели выбирается алгоритм (6 вариантов), вводятся целые координаты концов отрезка или центр и радиус окружности; ползунок и колесо мыши меняют масштаб (4–120 пикселей экрана на клетку) относительно позиции курсора, левая кнопка мыши сдвигает поле; кнопки «Вписать фигуру в окно» и «К началу координат» возвращают удобный вид. Флажки включают и отключают идеальную фигуру, линии сетки и номера пикселей в порядке построения. Поле рисует оси со стрелками и подписями x, y, числовые подписи с автоматическим подбором шага при смене масштаба, засечки и линии сетки; в строке состояния выводятся координаты клетки и точки плоскости под курсором. Вкладка «Сопроводительные вычисления» содержит текстовое пояснение формул и таблицу шагов выбранного алгоритма вместе с измеренным временем; вкладка «Временные характеристики» — таблицу замеров для всех алгоритмов. Кнопка «Сохранить отчёт» записывает вычисления в текстовый файл.

Привязка целых координат к сетке выводится на экран в левой панели: пиксель (x, y) — клетка с центром в точке (x, y); линии сетки проходят через полуцелые значения, а подписи на осях стоят под центрами клеток. Красная линия соединяет центры начальной и конечной клеток, поэтому наглядно видно, что алгоритм выбирает клетки, центры которых ближе всего к идеальной линии.

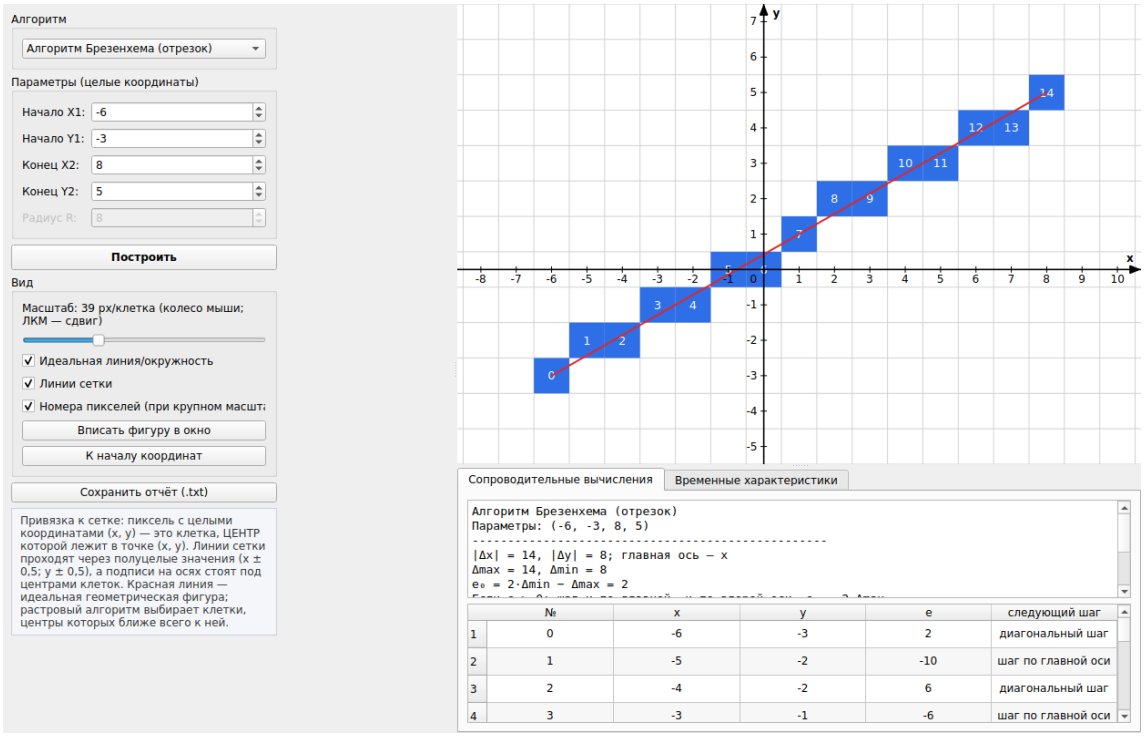


Рис. 1: Алгоритм Брезенхема (отрезок): пример $(-6, -3) \to (8, 5)$, таблица шагов, пиксели пронумерованы

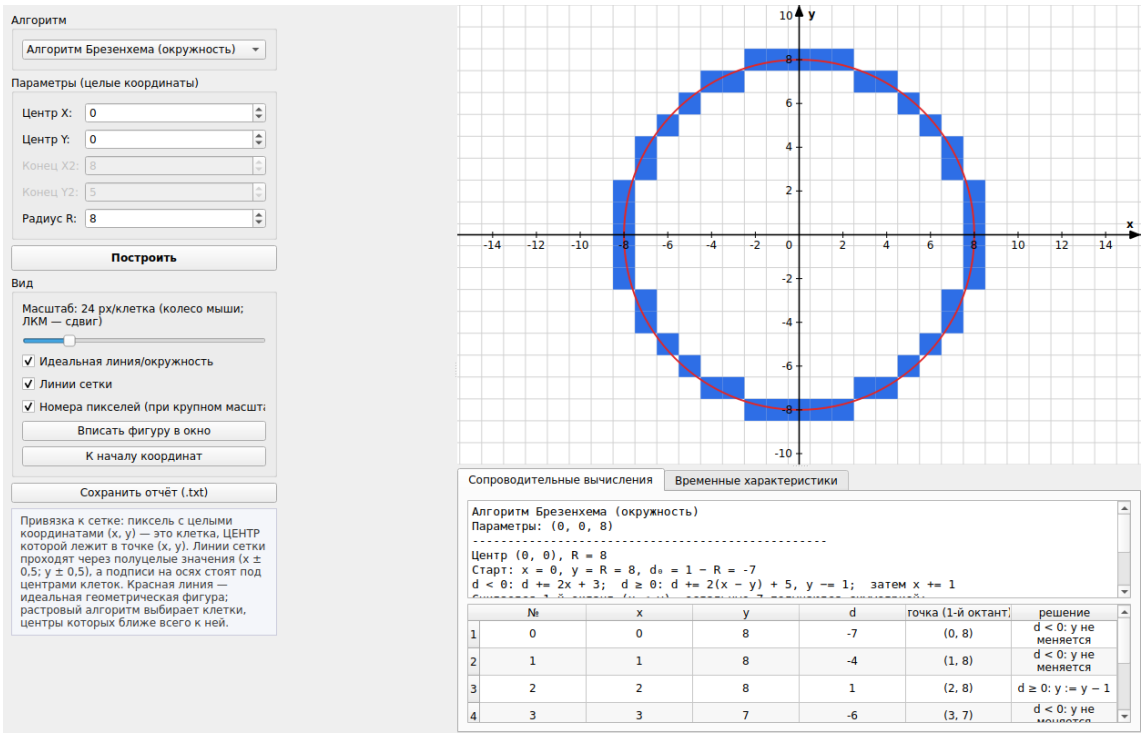


Рис. 2: Алгоритм Брезенхема (окружность): $R = 8$, центр $(0, 0)$

5. Анализ работы алгоритмов

5.1. Отрезки

5.1.1. Расчётный пример

Для отрезка $(-6, -3) \to (8, 5)$ имеем $\Delta x = 14$, $\Delta y = 8$, $k = \frac{4}{7} \approx 0{,}5714$, $b = y_1 - kx_1 = \frac{3}{7} \approx 0{,}4286$, $L = 14$, то есть должно получиться 15 пикселей. Все четыре целочисленных результата (пошаговый, ЦДА, Брезенхем, Кастл–Питвей) совпали и дали одну и ту же цепочку (рис. 3, левая часть); сглаживание по Bу даёт 27 пикселей (правая часть). Вычисления приведены в табл. 3.

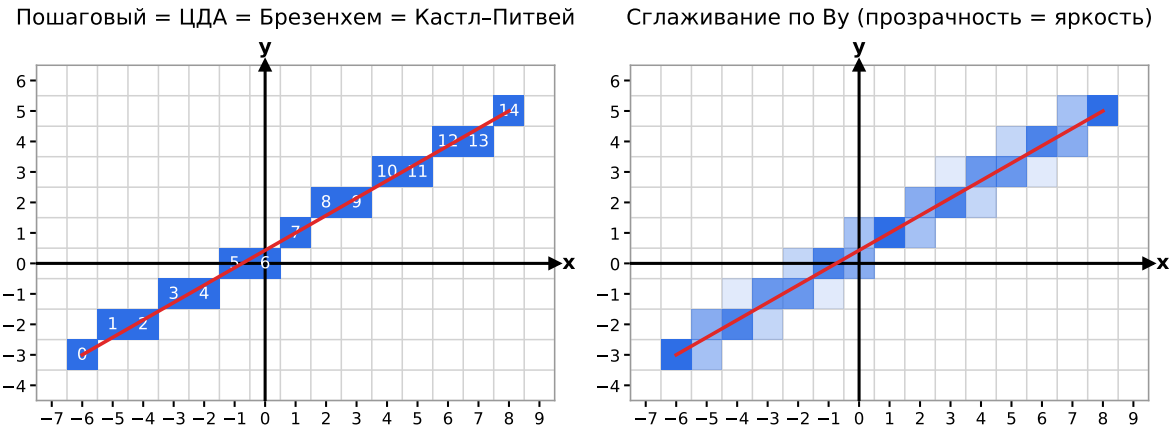


Рис. 3: Пример $(-6, -3) \to (8, 5)$: общий результат четырёх алгоритмов (слева; цифры — порядок построения) и сглаженная линия (справа)

Пошаговый алгоритм и ЦДА оперируют дробными величинами: в строке $i = 1$ значение $kx +$

Таблица 3: Расчёт примера $(-6,-3)\to(8,5)$ **(**$\Delta x=14$**,** $\Delta y=8$**,** $k=0{,}5714$**,** $b=0{,}4286$**)**

		Пошаговый		ЦДА		Брезенхем	
№	x	$kx+b$	y	y_i	Y	e_i	y
0	−6	−3,0000	−3	−3,0000	−3	2	−3
1	−5	−2,4286	−2	−2,4286	−2	−10	−2
2	−4	−1,8571	−2	−1,8571	−2	6	−2
3	−3	−1,2857	−1	−1,2857	−1	−6	−1
4	−2	−0,7143	−1	−0,7143	−1	10	−1
5	−1	−0,1429	0	−0,1429	0	−2	0
6	0	0,4286	0	0,4286	0	14	0
7	1	1,0000	1	1,0000	1	2	1
8	2	1,5714	2	1,5714	2	−10	2
9	3	2,1429	2	2,1429	2	6	2
10	4	2,7143	3	2,7143	3	−6	3
11	5	3,2857	3	3,2857	3	10	3
12	6	3,8571	4	3,8571	4	−2	4
13	7	4,4286	4	4,4286	4	14	4
14	8	5,0000	5	5,0000	5	2	5

$b=-2{,}4286$ округляется до -2, а в ЦДА то же значение получено накоплением $y_1=-3+0{,}5714$. Алгоритм Брезенхема вместо дробной ординаты хранит целый параметр e_i, знак которого показывает, выше или ниже середины между двумя клетками проходит линия. Например, $e_0=2\cdot 8-14=2\geq 0$ — диагональный шаг; $e_1=e_0+2\cdot 8-2\cdot 14=-10<0$ — шаг только по x; $e_2=-10+16=6\geq 0$ и так далее. Видно, что знак e_i предсказывает приращение y в следующей строке таблицы: $e_i\geq 0$ в строках $0,2,4,6,7,9,11,13,14$, и именно после них y возрастает (у последней строки следующего пикселя нет). В строке $x=1$ линия проходит ровно через центр пикселя ($kx+b=1{,}0000$).

Для алгоритма Кастл–Питвея $\gcd(14,8)=2$, $a'=7$, $b'=4$. Процесс построения периода: $(m_1,m_2)=(3,4)$, $s_1=$ `x`, $s_2=$ `d`; $m_1<m_2$: $s_1=$ `xd`, $(3,1)$; $m_1>m_2$: $s_2=$ `xdd`, $(2,1)$; $m_1>m_2$: $s_2=$ `xdxdd`, $(1,1)$. Период до сдвига $s_1s_2=$ `xdxdxdd` (3 шага `x` и 4 шага `d`), сдвиг $t=3\cdot 4^{-1}\bmod 7=3\cdot 2\bmod 7=6$ даёт период `dxdxdxd`; повторённый $g=2$ раза, он образует строку `dxdxdxddxdxdxd` из 14 шагов, совпадающую с приращениями y в табл. 3. Для сглаживания в табл. 4 приведены яркости: в каждой строке сумма двух яркостей равна 1, а в столбце $x=1$ вся яркость достаётся одному пикселю, так как линия проходит через его центр.

Таблица 4: Сглаживание по Ву, тот же пример (первые 8 столбцов из 15)

№	x	$y=kx+b$	y_{int}	$I(y_{\text{int}})$	$y_{\text{int}}+1$	$I(y_{\text{int}}+1)$
0	−6	−3,0000	−3	1,000	−2	0,000
1	−5	−2,4286	−3	0,429	−2	0,571
2	−4	−1,8571	−2	0,857	−1	0,143
3	−3	−1,2857	−2	0,286	−1	0,714
4	−2	−0,7143	−1	0,714	0	0,286
5	−1	−0,1429	−1	0,143	0	0,857
6	0	0,4286	0	0,571	1	0,429
7	1	1,0000	1	1,000	2	0,000

5.1.2. Все октанты и частные случаи

Рис. 4 показывает результат Брезенхема для 16 отрезков из начала координат: направления во всех восьми октантах и вдоль осей обработаны корректно, форма «звезды» симметрична. Кроме того, на 11 частных случаях (точка, горизонталь, вертикаль, диагонали в обе стороны, обратный

порядок концов) и на 5000 случайных отрезках для четырёх целочисленных алгоритмов проверены критерии (а): нарушений нет ни у одного алгоритма (число пикселей равно $L + 1$, концы совпадают с заданными, соседние пиксели соприкасаются).

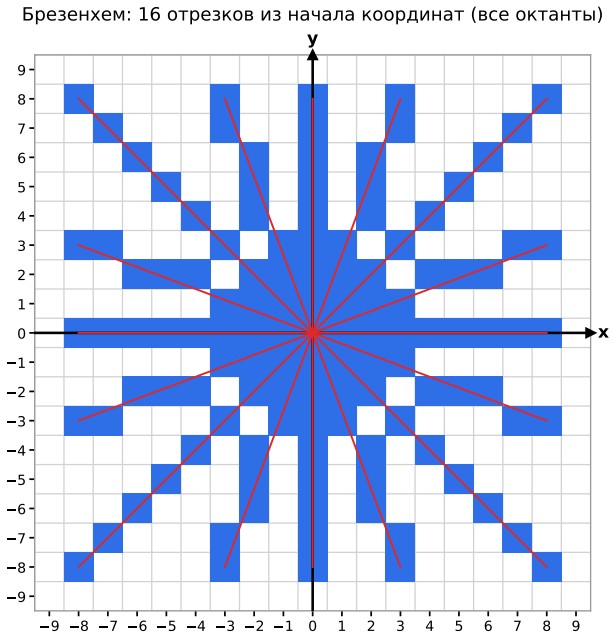


Рис. 4: Алгоритм Брезенхема: 16 отрезков из начала координат; красные линии — идеальные отрезки

5.1.3. Точность и согласованность алгоритмов

Результаты на 3000 случайных отрезках приведены в табл. 5. Все четыре алгоритма дают ровно $L + 1$ пиксель, максимальное отклонение по второстепенной оси равно $0{,}5$, а перпендикулярное расстояние до линии не превышает $0{,}5$. Среднее отклонение $0{,}2415$ близко к теоретическому $1/4$ (равномерная ошибка на $[-\frac{1}{2}, \frac{1}{2}]$). То есть *по точности четыре алгоритма эквивалентны*; различаются они в разрешении «ничьих», то есть на пикселях, где идеальная линия проходит ровно по границе двух клеток.

Таблица 5: Точность и согласованность на 3000 случайных отрезках (концы в $[-100, 100]^2$)

Алгоритм	$L{+}1$ пикс.	max $\lvert\varepsilon\rvert$	$\overline{\lvert\varepsilon\rvert}$	max ρ	Совпал с Брез.	Отличн. пикс.	$P{\to}Q \neq Q{\to}P$
Пошаговый	100 %	0,500	0,2415	0,500	83,0 %	0,57 %	4,5 %
ЦДА	100 %	0,500	0,2415	0,500	81,7 %	0,59 %	22,1 %
Брезенхем	100 %	0,500	0,2415	0,500	100,0 %	0,00 %	33,6 %
Кастл–Питвей	100 %	0,500	0,2415	0,500	100,0 %	0,00 %	33,6 %

Алгоритм Кастл–Питвея совпал с Брезенхемом на всех 3000 отрезках. Пошаговый алгоритм отличается от Брезенхема на $17{,}0\,\%$ отрезков, ЦДА — на $18{,}3\,\%$, но суммарно это только $0{,}57\,\%$ и $0{,}59\,\%$ пикселей. Все расхождения (1636 пикселей у пошагового и 1674 у ЦДА) приходятся на «ничьи»: точная проверка показала, что отклонение в этих пикселях равно ровно $1/2$, а всего таких пикселей в тестовом наборе $1{,}16\,\%$ и они встречаются на $33{,}6\,\%$ отрезков. Причин две. Во-первых, Брезенхем смещается на «ничьей» в направлении движения по второстепенной оси, а пошаговый алгоритм и ЦДА округляют всегда «половину вверх», то есть к $+\infty$. Эти правила совпадают, если вторая ось идёт в положительном направлении (из 517 таких отрезков с «ничьими» пошаговый алгоритм расходится с Брезенхемом только на 53), и расходятся, если

в отрицательном (456 из 490). Пример на рис. 5: для $(0,0) \to (10,-5)$ пиксели с нечётными номерами оказываются на разных сторонах. Во-вторых, пошаговый алгоритм и ЦДА работают с вещественными числами, и на «ничьей» результат определяется погрешностью округления ($kx+b$ может получиться $2{,}4999999\ldots$ вместо $2{,}5$): ЦДА отличается от точного округления «половина вверх» на 472 отрезках (15,7 %). Целочисленный Брезенхем таких проблем не имеет: его результат детерминирован.

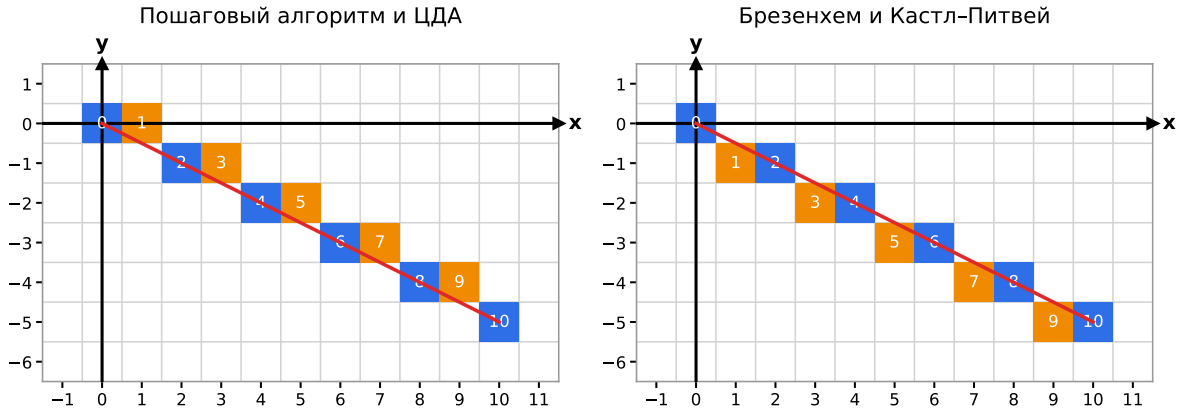


Рис. 5: Случай «ничьей»: отрезок $(0,0) \to (10,-5)$; оранжевым выделены пиксели, различающиеся между группами алгоритмов

Последний столбец табл. 5 показывает, что результат построения от P к Q и от Q к P у Брезенхема и Кастл–Питвея различается ровно на тех $33{,}6\,\%$ отрезков, где есть «ничьи» (на «ничьей» всегда выбирается направление движения, а оно при обращении отрезка меняется). Пошаговый алгоритм почти симметричен ($4{,}5\,\%$, только из-за погрешности вещественной арифметики), у ЦДА несимметричность $22{,}1\,\%$ из-за накопления погрешности по-разному в прямом и обратном направлении. Если важна идентичность пикселей при обращении отрезка (например, при отрисовке общей границы двух многоугольников), концы следует упорядочивать перед построением.

5.1.4. Накопление ошибки в ЦДА

Табл. 6 показывает, как растёт ошибка при накоплении приращения $\Delta y/L$ в одинарной точности (float32, как в аппаратных реализациях и шейдерах) и в двойной (float64, как в приложении). Эталон — точное значение $y_i = m\,i/n$.

Таблица 6: Накопление ошибки в ЦДА: отрезок $(0,0) \to (n, \lfloor 0{,}37n \rfloor + 1)$

n	Макс. ошибка y, float32	Неверных пикселей, float32	Неверных пикселей, float64
1 000	0,0017	1 (0,1 %)	0
10 000	0,1396	576 (5,8 %)	0
100 000	34,5072	88 182 (88,2 %)	0
1 000 000	2782,7812	987 362 (98,7 %)	1

В float32 уже при $n = 10^4$ неверно $5{,}8\,\%$ пикселей, при $n = 10^5$ отрезок уходит от идеальной линии на 34 пикселя, и построенная линия перестаёт быть отрезком. В float64 неверен один пиксель из 10^6: это пиксель, где идеальное значение отличается от границы клеток всего на 10^{-6}, и накопленной за $870\,000$ сложений погрешности достаточно, чтобы его перевернуть. Целочисленный Брезенхем ошибки не накапливает: параметр e_i вычисляется точно при любой длине (пока не переполняется целый тип).

5.2. Окружность

Рис. 6 и табл. 7 показывают построение окружности $R = 8$. Начало: $(x, y) = (0, 8)$, $d_0 = 1-8 = -7 < 0$, поэтому следующий пиксель $(1, 8)$, $d_1 = -7+2\cdot 0+3 = -4$; снова $d_1 < 0$, пиксель $(2, 8)$, $d_2 = -4+2\cdot 1+3 = 1$; теперь $d_2 \geq 0$, поэтому y уменьшается: $(3, 7)$ и $d_3 = 1+2(2-8)+5 = -6$, и так далее до диагонали $x = y$ в точке $(5, 6)$; с учётом симметрии получается 44 пикселя.

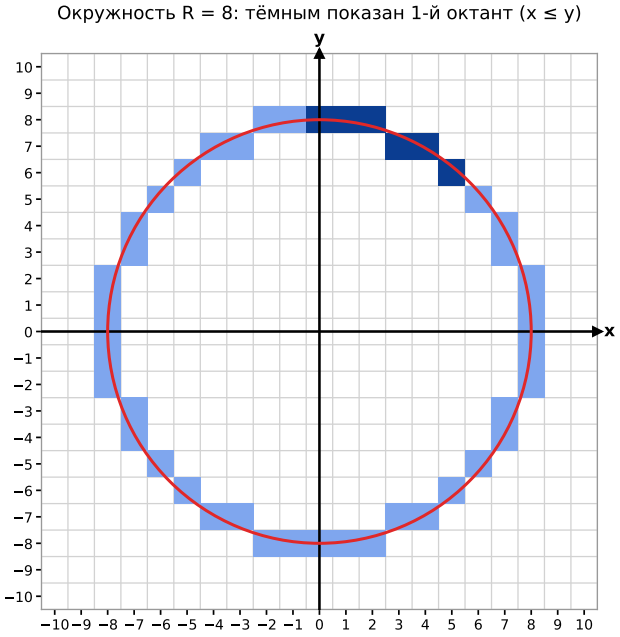


Рис. 6: Окружность $R = 8$: тёмным выделен 1-й октант, остальные пиксели получены отражениями

Таблица 7: Окружность $R = 8$ с центром $(0, 0)$: 1-й октант ($d_0 = 1 - R = -7$)

№	x	y	d	Решение	Пиксель
0	0	8	−7	$d < 0$: y не меняется	(0, 8)
1	1	8	−4	$d < 0$: y не меняется	(1, 8)
2	2	8	1	$d \geq 0$: $y := y - 1$	(2, 8)
3	3	7	−6	$d < 0$: y не меняется	(3, 7)
4	4	7	3	$d \geq 0$: $y := y - 1$	(4, 7)
5	5	6	2	$d \geq 0$: $y := y - 1$	(5, 6)

Точность подтверждается для всех $R = 1, \ldots, 300$: максимальное радиальное отклонение $\left|\sqrt{x^2 + y^2} - R\right|$ равно $0{,}4987 < 0{,}5$ (табл. 8); число пикселей близко к $4\sqrt{2}\,R \approx 5{,}66R$. Множества пикселей реализованной формы ($d_0 = 1 - R$) и классической ($\Delta_0 = 3 - 2R$) совпали для всех 300 радиусов, а также с эталонами «ближайший пиксель по радиальному расстоянию» и «ближайший по вертикали в каждом столбце октанта», так что алгоритм выбирает ближайшие к окружности пиксели. Частные случаи корректны: $R = 0$ даёт одну точку, $R = 1$ — четыре, $R = 2$ — двенадцать пикселей.

5.3. Сглаживание по Ву и алгоритм Кастл–Питвея

Сглаживание. На всех 3000 случайных отрезках сумма яркостей в каждом столбце равна 1 с погрешностью $1{,}4 \cdot 10^{-12}$, а яркостный центр столбца отстоит от идеальной линии не более чем на $1{,}4 \cdot 10^{-10}$ пикселя (это погрешность округления вещественных чисел), тогда как у несглаженной

Таблица 8: Окружность: число пикселей и максимальное радиальное отклонение $\max\left|\sqrt{x^2+y^2}-R\right|$

R	Пикселей	Пикселей/R	Макс. отклонение, пикс.
1	4	4,00	0,000
2	12	6,00	0,236
5	28	5,60	0,385
8	44	5,50	0,384
20	112	5,60	0,396
100	564	5,64	0,496
300	1696	5,65	0,494

линии отклонение достигает $0{,}5$ (среднее $0{,}24$). Визуальный эффект показан на рис. 7: ступеньки у линии с наклоном $9/31$ исчезают. Цена — в среднем около $1{,}8$ закрашенных пикселя на столбец при наклоне $0{,}6$ и необходимость смешения с фоном (в приложении это альфа-смешение над белым). Смешение происходит в пространстве sRGB без гамма-коррекции, поэтому линия выглядит чуть темнее и толще, чем при линейном по энергии смешении.



Рис. 7: Отрезок $(0,0)\to(31,9)$**: Брезенхем (слева) и Ву (справа); изображения не интерполировались**

Кастл–Питвей. Алгоритм совпал с Брезенхемом пиксель в пиксель на всех случайных отрезках и на всех отрезках $(0,0)\to(a,b)$, $0<b<a<25$; формула сдвига (9) дополнительно проверена для всех $0<b<a<120$. Строка шагов строится заранее; когда НОД велик, период a' короткий, и строка дёшево повторяется. Если же $\gcd(a,b)=1$, то $a'=a$, и построение периода — это тот же объём работы, что и сам отрезок; выигрыш тогда достигается только за счёт повторного использования строки для многих отрезков одного наклона.

5.4. Время работы

Измерялся один вызов функции растеризации (без отрисовки и без заполнения таблицы вычислений). Время — минимум из 11 раундов, в каждом из которых алгоритмы вызывались по очереди по $\max(5,\ 400\,000/n)$ раз; такая схема уменьшает влияние шума виртуальной машины (между двумя независимыми запусками результаты расходились не более чем на 3–4 %). Тестовая машина: Intel Xeon 2,1 ГГц (1 виртуальное ядро), Linux, Python 3.12.3. Отрезок — $(0,0)\to(n,\lfloor 0{,}6n\rfloor)$, окружность — радиус $R=n$. Абсолютные значения зависят от процессора и интерпретатора; в самом приложении время показывается для текущего примитива и усредняется по 300 запускам, а вкладка «Временные характеристики» воспроизводит табл. 9 (с разбросом до 10–20 % между запусками, так как замер там одиночный).

Из табл. 9 и рис. 8 следует:

Таблица 9: Время одного вызова (мкс), n — длина отрезка по главной оси или радиус

Алгоритм	n=10	n=100	n=10^3	n=10^4	нс/пиксель (n=10^4)	отн. к Брезенхему
Пошаговый	1,7	10,8	112	1 532	153	1,10
ЦДА	2,1	13,7	142	1 812	181	1,31
Брезенхем (отрезок)	1,2	8,4	104	1 387	139	1,00
Брезенхем (окружность)	9,5	88,1	1 404	13 882	245	—
Кастл–Питвей	1,6	7,0	77,9	1 103	110	0,80
Ву (сглаживание)	2,2	16,0	179	2 760	153	1,99

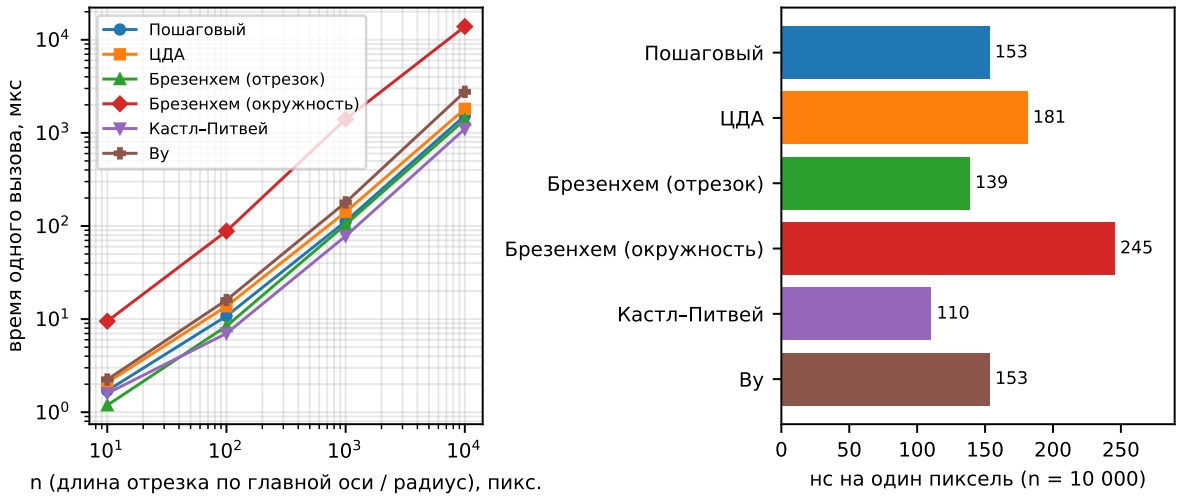


Рис. 8: Время работы в зависимости от n (слева) и стоимость одного пикселя при $n = 10^4$ (справа)

- Все алгоритмы линейны по размеру задачи: при росте n с 10^2 до 10^3 и далее до 10^4 время растёт в 10–16 раз, а между $n = 10$ и $n = 100$ — лишь в 4–9 раз из-за постоянных накладных расходов на вызов функции и создание списков.
- Пошаговый алгоритм, ЦДА и Брезенхем отличаются не более чем на 31 %: пошаговый алгоритм на 10 % медленнее Брезенхема, ЦДА — на 31 %. Преимущество целых чисел над вещественными в интерпретируемом Python сильно сглажено, поскольку любая операция стоит десятки наносекунд независимо от типа; поэтому различие определяется числом операций: у ЦДА их больше (два накапливаемых значения и два округления), у Брезенхема — меньше.
- Кастл–Питвей быстрее Брезенхема на 20 % (в цикле только перебор строки и инкременты). Наклон тестовых отрезков $3/5$ даёт очень короткий период ($a' = 5$), что выгодно этому алгоритму. В отдельной проверке для $(0, 0) \to (10^4, 6001)$ с $\gcd = 1$ (период равен длине отрезка) выигрыш сохранился, но уменьшился примерно до 15–20 % (однократный замер, менее надёжный): преимущество в нашей реализации связано прежде всего с простотой цикла, а не с периодичностью шаблона.
- Сглаживание по Ву примерно вдвое медленнее Брезенхема: на каждый столбец приходится около $1{,}8$ пикселя и вещественные вычисления. В пересчёте на один закрашенный пиксель оно стоит примерно столько же, сколько пошаговый алгоритм (153 нс), но пикселей вдвое больше.
- Окружность дороже в пересчёте на пиксель (245 нс): на каждый шаг цикла приходится 8 отражений и проверка на дубликаты, но на один шаг октанта выполняется всего $\approx 1{,}96$ мкс. Сравнивать абсолютные времена отрезка и окружности нет смысла: это разные примитивы.

Порядок алгоритмов и величина разрыва относятся к данной реализации на Python: в компилируемом языке, где вещественное умножение и округление заметно дороже целого сложения, разрыв между ЦДА, пошаговым и Брезенхемом мог бы выглядеть иначе, и это не проверялось.

6. Выводы

Количественная проверка на наборе примитивов позволяет сформулировать следующие выводы.

1. **Общая корректность.** Все реализованные целочисленные алгоритмы для отрезков дают $L+1$ связный пиксель с отклонением не более $1/2$ пикселя от идеальной линии; различия между ними сводятся к разрешению «ничьих» (идеальная линия ровно на границе клеток, $1{,}16\,\%$ пикселей). Поэтому выбирать алгоритм следует не по точности, а по составу вычислений, детерминированности и требованиям к качеству изображения.
2. **Пошаговый алгоритм.**
   - *Особенности:* прямое применение уравнения $y = kx + b$; для $|k| > 1$ и вертикальных линий нужна смена осей; ошибка не накапливается, так как каждое y считается заново.
   - *Плюсы:* максимальная простота и наглядность, легко обобщается на графики функций $y = f(x)$.
   - *Минусы:* вещественное умножение и округление на каждом пикселе, чувствительность результата на «ничьих» к погрешности округления (отличается от точного округления в части случаев), на $10\,\%$ медленнее Брезенхема в нашей реализации.
   - *Применение:* учебные задачи, прототипы, построение графиков, где скорость не критична.
3. **ЦДА.**
   - *Особенности:* единая схема для любого наклона, шаг по главной оси ровно 1, второй координатой управляет накопление приращения $\Delta y/L$.
   - *Плюсы:* нет умножения в цикле; подходит для аппаратной реализации и для интерполяции атрибутов (цвета, глубины) вдоль линии.
   - *Минусы:* вещественная арифметика; ошибка накапливается пропорционально длине и зависит от разрядности (при float32 неверно $5{,}8\,\%$ пикселей уже при $n = 10^4$ и 34 пикселя смещения при $n = 10^5$); результат на «ничьих» зависит от погрешности округления; самый медленный из трёх базовых алгоритмов ($+31\,\%$ к Брезенхему); построение от P к Q и обратно совпадает лишь в $78\,\%$ случаев.
   - *Применение:* короткие отрезки при float64 или арифметике с фиксированной запятой с достаточным числом дробных разрядов; интерполяция параметров вдоль отрезка.
4. **Алгоритм Брезенхема (отрезок).**
   - *Особенности:* вещественные ординаты заменены целым параметром e_i, знак которого определяет, выбирать ли верхнего соседа.
   - *Плюсы:* только целые сложения и сравнение; ошибка не накапливается при любой длине; точно детерминирован; минимальное отклонение ($\leq 1/2$); быстрее пошагового и ЦДА.
   - *Минусы:* ветвление в цикле; требуется приведение к 1-му октанту; нет сглаживания; на «ничьей» результат зависит от направления построения (на $33{,}6\,\%$ отрезков построение от P к Q и обратно даёт разные пиксели); линия единичной толщины.
   - *Применение:* основной алгоритм для линий единичной толщины без сглаживания: интерфейсы, САПР, плоттеры, встраиваемые системы без вещественных вычислений.

5. **Алгоритм Брезенхема (окружность).**

   - *Особенности:* строится только 1/8 дуги, остальные пиксели — отражениями; решение по знаку функции F в средней точке.
   - *Плюсы:* целые числа; отклонение от окружности не более $0{,}4987$ для $R \leq 300$; совпадает с классической формой $\Delta_0 = 3 - 2R$ и с «ближайшим пикселем»; примерно $5{,}66R$ пикселей без разрывов.
   - *Минусы:* только окружность с целыми центром и радиусом (эллипсы, дуги и произвольные кривые требуют других вариантов алгоритма); на осях и диагоналях отражения дублируют пиксели, и их нужно отфильтровать; единичная толщина, без сглаживания.
   - *Применение:* окружности и дуги в интерфейсах, играх и САПР; как основа для скруглений в растровых редакторах.

6. **Алгоритм Кастл–Питвея.**

   - *Особенности:* шаблон шагов строится один раз как слово по алгоритму Евклида над строками, затем циклически сдвигается для округления к ближайшему пикселю и повторяется $\gcd(a, b)$ раз.
   - *Плюсы:* пиксель в пиксель совпадает с Брезенхемом; в цикле нет вычисления ошибки; один шаблон можно использовать для всех отрезков с одним наклоном; в нашей реализации на 20 % быстрее Брезенхема.
   - *Минусы:* сложнее для понимания и реализации (НОД, обратный элемент по модулю для сдвига); требует памяти $O(a')$ на строку шагов; при $\gcd(a, b) = 1$ выигрыш от периодичности отсутствует.
   - *Применение:* массовое рисование линий с повторяющимися наклонами (штриховка, сетки, шаблоны), быстрые реализации, где ветвление дорого.

7. **Сглаживание по Ву.**

   - *Особенности:* вместо выбора одной клетки закрашиваются две; яркость обратно пропорциональна расстоянию до линии, сумма в столбце равна 1.
   - *Плюсы:* исчезают ступеньки, одинаковая яркость вдоль линии, субпиксельная точность положения (центр яркости совпадает с линией с погрешностью около 10^{-10} пикселя); линия не требует особой обработки концов при целых координатах.
   - *Минусы:* примерно вдвое медленнее Брезенхема; требует смешения с фоном (не годится для индексированных палитр без дополнительной обработки); без гамма-коррекции яркость линии искажена; линия кажется «размытой» и менее чёткой.
   - *Применение:* графика высокого качества: диаграммы, векторные интерфейсы, тонкие линии на экранах высокого разрешения.

8. **Рекомендации по выбору.** Линии без сглаживания — Брезенхем; много линий с одинаковым наклоном — Кастл–Питвей; интерполяция вдоль линии и аппаратная реализация — ЦДА (в float64 или с достаточным числом дробных разрядов); окружности — Брезенхем для окружности; линии, которые должны выглядеть гладко, — Ву; для объяснения идеи растеризации — пошаговый.

9. **Ограничения работы.** Время измерено на одной виртуальной машине в интерпретируемом Python, поэтому относительные различия в 10–30 % между пошаговым, ЦДА и Брезенхемом не следует переносить на другие языки и процессоры; статистика точности получена на 3000 случайных отрезках и 300 окружностях; не реализованы отсечение, толщина линий и штриховые стили; обработка окружности ограничена целыми центром и радиусом. Приложение использует PyQt6, поэтому работает под Windows 10 и новее, а также под Linux и

macOS; на Windows XP оно не запустится (для XP потребовалась бы иная связка — например, более старая версия Python и Qt, либо веб-версия).

7. Ссылки

- Исходный код приложения (файл `raster_lab3.py`): `https://github.com/Radith-c-like/BSU_ComputerGraphics`.
- Bresenham J. E. Algorithm for computer control of a digital plotter // IBM Systems Journal, 1965, vol. 4, no. 1, pp. 25–30.
- Bresenham J. E. A linear algorithm for incremental digital display of circular arcs // Communications of the ACM, 1977, vol. 20, no. 2, pp. 100–106.
- Pitteway M. L. V. Algorithm for drawing ellipses or hyperbolae with a digital plotter // The Computer Journal, 1967, vol. 10, no. 3, pp. 282–289.
- Brons R. Linguistic methods for the description of a straight line on a grid // Computer Graphics and Image Processing, 1974, vol. 3, no. 1, pp. 48–62.
- Castle W. D. F., Pitteway M. L. V. An application of Euclid’s algorithm to drawing straight lines // Fundamental Algorithms for Computer Graphics (ed. R. A. Earnshaw), NATO ASI Series F, vol. 17, Springer, 1985, pp. 135–139.
- Wu X. An efficient antialiasing technique // Computer Graphics (SIGGRAPH ’91), 1991, vol. 25, no. 4, pp. 143–152.
- Newman W. M., Sproull R. F. Principles of Interactive Computer Graphics. 2nd ed. McGraw-Hill, 1979 (цифровой дифференциальный анализатор).
- Foley J. D., van Dam A., Feiner S. K., Hughes J. F. Computer Graphics: Principles and Practice. 2nd ed. Addison-Wesley, 1990 (гл. 3: растровые алгоритмы).
- Hearn D., Baker M. P. Computer Graphics with OpenGL. 4th ed. Pearson, 2011.
- Riverbank Computing. PyQt6 Documentation: `https://www.riverbankcomputing.com/static/Docs/PyQt6/`.
- Hunter J. D. Matplotlib: a 2D graphics environment // Computing in Science & Engineering, 2007, vol. 9, no. 3, pp. 90–95.